

Lung Cancer Dataset – Data Exploration & Preprocessing

Presented to

Faculty Dean: Dr . Mohamed Talaat.

Dr: Rasha Saleh

TA: Mohamed Tarek

Participants

Hosny Hassan Hosny 192200372

Hesham Mansour 191900312

1. Introduction

In the context of data mining, the initial phase of data exploration and preprocessing plays a vital role in ensuring the reliability and effectiveness of subsequent analysis and modeling. This report documents the first stage of a comprehensive data mining project focused on a publicly available lung cancer dataset. The primary objective at this stage is to thoroughly understand the structure and content of the dataset, identify and address data quality issues such as missing values and outliers, and apply appropriate preprocessing techniques. These foundational steps are essential to prepare the dataset for more advanced data mining tasks and to facilitate the extraction of meaningful insights.

This project focuses on the ****exploration and preprocessing**** of the [Lung Cancer Dataset](<https://www.kaggle.com/datasets/khwaishsaxena/lung-cancer-dataset>). It is part of a final project task that includes cleaning, transforming, and preparing the dataset for future machine learning use.

2. Dataset Overview

The dataset consists of medical data related to patients who may have lung cancer. Each instance in the dataset represents a patient, with various features representing health indicators and risk factors.

Features Include:

- GENDER (Male/Female)
- AGE
- SMOKING
- YELLOW_FINGERS
- ANXIETY
- PEER_PRESSURE
- CHRONIC_DISEASE
- FATIGUE
- ALLERGY
- WHEEZING

- ALCOHOL CONSUMING
- COUGHING
- SHORTNESS OF BREATH
- SWALLOWING DIFFICULTY
- CHEST PAIN
- LUNG_CANCER (Target: Yes/No)

1. Libraries and Tools Used:

```
1 import pandas as pd #used for data manipulation and analysis. It provides data structures like DataFrames.
2 import numpy as np # numerical computations in Python. It provides support for arrays, matrices, and mathematical functions.
3 import matplotlib.pyplot as plt #used for creating static, interactive, and animated visualizations.
4 import seaborn as sns # a statistical data visualization library built on Matplotlib. It provides a high-level interface for drawing attractive graphs.
5 # scikit-learn library, which is a popular tool for machine learning in Python:
6 from sklearn.model_selection import train_test_split #Used to split datasets into training and testing subsets for model evaluation.
7 from sklearn.preprocessing import MinMaxScaler, StandardScaler, OneHotEncoder # A preprocessing tool to standardize features by removing the mean and scaling to unit variance.
8 #Used to convert categorical variables into a one-hot encoded numeric array.
9 from sklearn.impute import SimpleImputer #Used to handle missing values by imputing them (e.g., replacing NaN with mean/median/mode).
10 from sklearn.compose import ColumnTransformer #Applies different transformations to different columns in a dataset (e.g., scaling numerical columns and one-hot encoding categorical columns).
11 from sklearn.pipeline import Pipeline #Chains multiple data processing steps (e.g., imputation, scaling, encoding) into a single object for streamlined workflow.
```

2. Load the dataset:

- Dataset loaded using `pandas.read_csv()`
- Handled missing values by detecting `'?'` as `'NaN'`

```
1
2 file_path = r"D:\data mining\task1\data-mining-project\Lung Cancer.csv" # Use actual file name
3 df = pd.read_csv(file_path)
```

3. Explore the dataset's features and target variable

- Displayed first 5 rows
- Inspected datatypes and value distributions
- Examined missing values per column

```
1 print("First 5 rows of the dataset:")
2 print(df.head())
3 print("Understand datatypes and nulls:")
4 print(df.info())
5 print("Summary statistics:")
6 print(df.describe())
7 print("Explore the target distribution:")
8 print(df['survived'].value_counts())
```

Output :

```
''' First 5 rows of the dataset:
   id  age  gender  country  diagnosis_date  cancer_stage  family_history \
0    1  64.0   Male   Sweden    2016-04-05    Stage I        Yes
1    2  50.0  Female Netherlands    2023-04-20    Stage III       Yes
2    3  65.0  Female   Hungary    2023-04-05    Stage III       Yes
3    4  51.0  Female   Belgium    2016-02-05    Stage I         No
4    5  37.0   Male  Luxembourg    2023-11-29    Stage I         No

   smoking_status  bmi  cholesterol_level  hypertension  asthma  cirrhosis \
0  Passive Smoker  29.4                199              0        0        1
1  Passive Smoker  41.2                280              1        1        0
2   Former Smoker  44.0                268              1        1        0
3  Passive Smoker  43.0                241              1        1        0
4  Passive Smoker  19.7                178              0        0        0
```

```

other_cancer treatment_type end_treatment_date survived
0          0      Chemotherapy      2017-09-10          0
1          0          Surgery      2024-06-17          1
2          0      Combined      2024-04-09          0
3          0      Chemotherapy      2017-04-23          0
4          0      Combined      2025-01-08          0

```

```

Understand datatypes and nulls:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 890000 entries, 0 to 889999
Data columns (total 17 columns):
...
survived
0      693996
1      196004
Name: count, dtype: int64

```

4. Handle missing values and outliers

- Identified and filled missing values using:
 - ****Median**** for numeric features
 - ****Mode**** for categorical features (optional)

```
1 print(df.isnull().sum())
```

Output :

```

id          0
age         0
gender      0
country     0
diagnosis_date  0
cancer_stage  0
family_history  0
smoking_status  0
bmi         0
cholesterol_level  0
hypertension  0
asthma      0
cirrhosis   0
other_cancer  0
treatment_type  0
end_treatment_date  0
survived     0
dtype: int64

```

5. Outlier Detection and Handling:

- Used the **IQR method** to detect outliers in numeric columns
- Applied two methods:
 - **Removal** of rows containing outliers
 - **Capping (Winsorization)** to clip outlier values to valid range.

```
1 # def detect_outliers_iqr(df, column):
2 #     Q1 = df[column].quantile(0.25)
3 #     Q3 = df[column].quantile(0.75)
4 #     IQR = Q3 - Q1
5 #     lower_bound = Q1 - 1.5 * IQR
6 #     upper_bound = Q3 + 1.5 * IQR
7 #     outliers = df[(df[column] < lower_bound) | (df[column] > upper_bound)]
8 #     return outliers
9
10 # # Example: detect outliers in 'bmi'
11 # bmi_outliers = detect_outliers_iqr(df, 'bmi')
12 # print("BMI Outliers:")
13 # print(bmi_outliers[['id', 'bmi']])
14
15
16 def detect_outliers_iqr(df):
17     numeric_cols = df.select_dtypes(include='number').columns
18     outlier_summary = {}
19
20     for col in numeric_cols:
21         Q1 = df[col].quantile(0.25)
22         Q3 = df[col].quantile(0.75)
23         IQR = Q3 - Q1
24         lower_bound = Q1 - 1.5 * IQR
25         upper_bound = Q3 + 1.5 * IQR
26
27         outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]
28         outlier_count = outliers.shape[0]
29
30         if outlier_count > 0:
31             outlier_summary[col] = {
32                 'count': outlier_count,
33                 'lower_bound': lower_bound,
34                 'upper_bound': upper_bound,
35                 'outliers': outliers[[col]]
36             }
37
38     return outlier_summary
39
40 # Run detection
41 outliers_by_column = detect_outliers_iqr(df)
42
43 # Print summary
44 for col, info in outliers_by_column.items():
45     print(f"\nColumn: {col}")
46     print(f"Number of outliers: {info['count']}")
47     print(f"Lower bound: {info['lower_bound']:.2f}, Upper bound: {info['upper_bound']:.2f}")
48     print(info['outliers'])
```

Output:

```
Column: age
Number of outliers: 3895
Lower bound: 27.00, Upper bound: 83.00
```

	age
23	21.0
477	85.0
576	87.0
755	25.0
760	90.0
...	...
889121	26.0
889411	87.0
889611	84.0
889771	84.0
889858	87.0

```
[3895 rows x 1 columns]
```

```
Column: hypertension
Number of outliers: 222479
Lower bound: 1.00, Upper bound: 1.00
```

	hypertension
0	0
4	0

```
...
```

889990	1
889997	1

Remove Outliers:

```
1 def remove_outliers(df):
2     numeric_cols = df.select_dtypes(include='number').columns
3     for col in numeric_cols:
4         Q1 = df[col].quantile(0.25)
5         Q3 = df[col].quantile(0.75)
6         IQR = Q3 - Q1
7         lower = Q1 - 1.5 * IQR
8         upper = Q3 + 1.5 * IQR
9         df = df[(df[col] >= lower) & (df[col] <= upper)]
10    return df
11
12 df_no_outliers = remove_outliers(df.copy())
13 print(df_no_outliers.head())
```

output:

	id	age	gender	country	diagnosis_date	cancer_stage	family_history	\
2	3	65.0	Female	Hungary	2023-04-05	Stage III	Yes	
3	4	51.0	Female	Belgium	2016-02-05	Stage I	No	
5	6	50.0	Male	Italy	2023-01-02	Stage I	No	
7	8	51.0	Male	Denmark	2017-02-18	Stage IV	Yes	
14	15	67.0	Female	Poland	2023-04-12	Stage II	Yes	

	smoking_status	bmi	cholesterol_level	hypertension	asthma	cirrhosis	\
2	Former Smoker	44.0	268	1	1	0	
3	Passive Smoker	43.0	241	1	1	0	
5	Never Smoked	37.6	274	1	0	0	
7	Former Smoker	25.8	195	1	1	0	
14	Former Smoker	42.7	259	1	1	0	

	other_cancer	treatment_type	end_treatment_date	survived
2	0	Combined	2024-04-09	0
3	0	Chemotherapy	2017-04-23	0
5	0	Radiation	2024-12-27	0
7	0	Combined	2017-08-26	0
14	0	Radiation	2024-02-18	0

6. Perform feature scaling or normalization

- Applied **Min-Max Normalization** to scale numeric features between 0 and 1 using `MinMaxScaler`
- Standardization using `StandardScaler` is supported (not the default)

```
1 numeric_cols = df.select_dtypes(include='number').columns
2 scaler = MinMaxScaler()
3 df_normalized = df.copy()
4 df_normalized[numeric_cols] = scaler.fit_transform(df[numeric_cols])
5 print(df_normalized[numeric_cols].describe())
```

Output :

	id	age	bmi	cholesterol_level	\
count	890000.000000	890000.000000	890000.000000	890000.000000	
mean	0.500000	0.510070	0.499799	0.557559	
std	0.288676	0.099945	0.288570	0.289549	
min	0.000000	0.000000	0.000000	0.000000	
25%	0.250000	0.440000	0.251724	0.306667	
50%	0.500000	0.510000	0.500000	0.613333	
75%	0.750000	0.580000	0.748276	0.806667	
max	1.000000	1.000000	1.000000	1.000000	

	hypertension	asthma	cirrhosis	other_cancer	\
count	890000.000000	890000.000000	890000.000000	890000.000000	
mean	0.750024	0.469740	0.225956	0.088157	
std	0.432999	0.499084	0.418211	0.283524	
min	0.000000	0.000000	0.000000	0.000000	
25%	1.000000	0.000000	0.000000	0.000000	
50%	1.000000	0.000000	0.000000	0.000000	
75%	1.000000	1.000000	0.000000	0.000000	
max	1.000000	1.000000	1.000000	1.000000	

	survived
count	890000.000000
mean	0.220229
std	0.414401
min	0.000000
25%	0.000000
50%	0.000000
75%	0.000000
max	1.000000

7. Split the dataset into training and testing sets:

- Separated data into **features ('X')** and **target ('y')**
- Encoded categorical variables using `pd.get_dummies()`
- Split into training and testing sets using `train_test_split()` with `stratify=y` for balanced classes

```

1 # 2. Define features (X) and target (y)
2 X = df.drop('survived', axis=1) # All columns except 'survived'
3 y = df['survived']             # The target column
4
5 # 3. Encode categorical features
6 # X = pd.get_dummies(X, drop_first=True)
7
8 X_train, X_test, y_train, y_test = train_test_split(
9     X, y,
10     test_size=0.2,           # 20% for testing, 80% for training
11     random_state=42,         # Ensures reproducibility
12     stratify=y               # Keeps class distribution balanced (important for classification)
13 )
14 print("Training set:", X_train.shape)
15 print("Testing set:", X_test.shape)
16

```

Output :

Training set: (712000, 16)

Testing set: (178000, 16)

8. Feature Engineering:

- Created a new column: `treatment\_duration\_days` by calculating:

...

treatment_end_date - diagnosis_date

...

- Visualized this feature using a **boxplot** to analyze its spread and detect any anomalies

```
1 # engineer a new column that calculates how long the patient was under treatment:
2 df['diagnosis_date'] = pd.to_datetime(df['diagnosis_date'])
3 df['end_treatment_date'] = pd.to_datetime(df['end_treatment_date'])
4
5 df['treatment_duration_days'] = (df['end_treatment_date'] - df['diagnosis_date']).dt.days
6
7 print(df[['id', 'diagnosis_date', 'end_treatment_date', 'treatment_duration_days']].head())
```

Output:

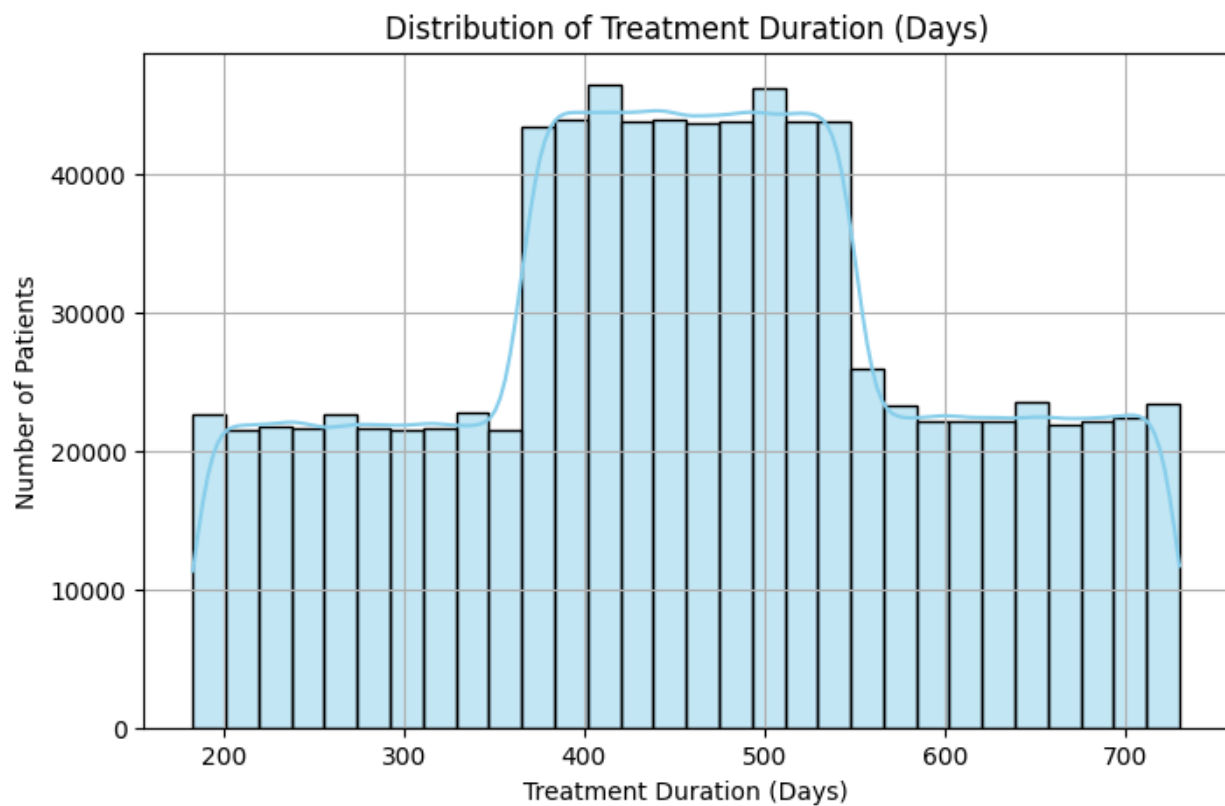
	id	diagnosis_date	end_treatment_date	treatment_duration_days
0	1	2016-04-05	2017-09-10	523
1	2	2023-04-20	2024-06-17	424
2	3	2023-04-05	2024-04-09	370
3	4	2016-02-05	2017-04-23	443
4	5	2023-11-29	2025-01-08	406

9. Visualizations:

- Boxplot of `treatment\_duration\_days` to analyze distribution
- (Optional) Histogram and additional plots can be added for deeper insights

```
1 plt.figure(figsize=(8, 5))
2 sns.histplot(df['treatment_duration_days'].dropna(), bins=30, kde=True, color="skyblue")
3 plt.title("Distribution of Treatment Duration (Days)")
4 plt.xlabel("Treatment Duration (Days)")
5 plt.ylabel("Number of Patients")
6 plt.grid(True)
7 plt.show()
```

Output:



```
1 plt.figure(figsize=(6, 4))
2 sns.boxplot(x=df['treatment_duration_days'], color="lightcoral")
3 plt.title("Boxplot of Treatment Duration")
4 plt.xlabel("Treatment Duration (Days)")
5 plt.grid(True)
6 plt.show()
```

Output:

